



TMDB Movie Pipeline

End-to-End Data Pipeline

Bronze → Silver → Gold → Dashboard

By : Nabila Hulwana Z.

Final Project Bootcamp Data Engineering rubythelib.ai

Project in github : [CLICK HERE](#)

Overview

Project Context

Building an end-to-end data pipeline to analyze the global film industry using Medallion Architecture.

Key Questions

1. What are the revenue/profit trends in the film industry from 1992–2016?
2. Which genre has been consistently the most profitable?
3. Which director has the highest ROI throughout their career?
4. Which films fall into the Mega Blockbuster category (very high performance)?

Data Description (Secondary-dataset)

1 TMDb API

- Data Type : JSON
- source : [klik ini](#)
- 80 rows
- API Key, Endpoint/ movie/ popular

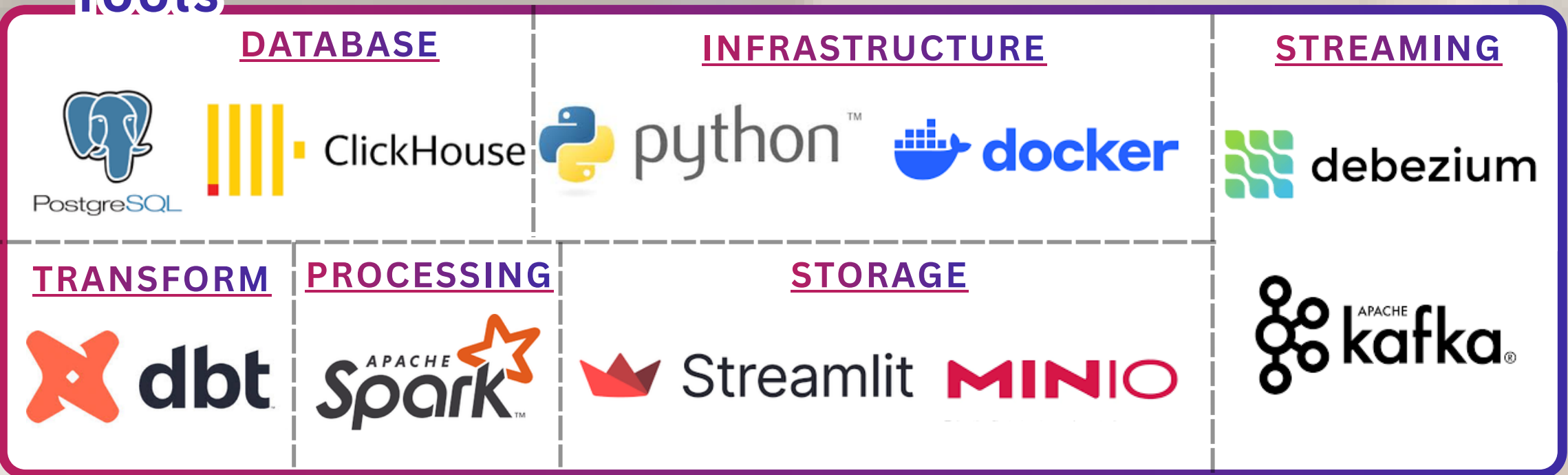
2 Kaggle – tmdb_5000_movies.csv

- Data type : CSV
- Source : [klik ini](#)
- 4673 rows
- Budget, revenue, genres, keywords, companies

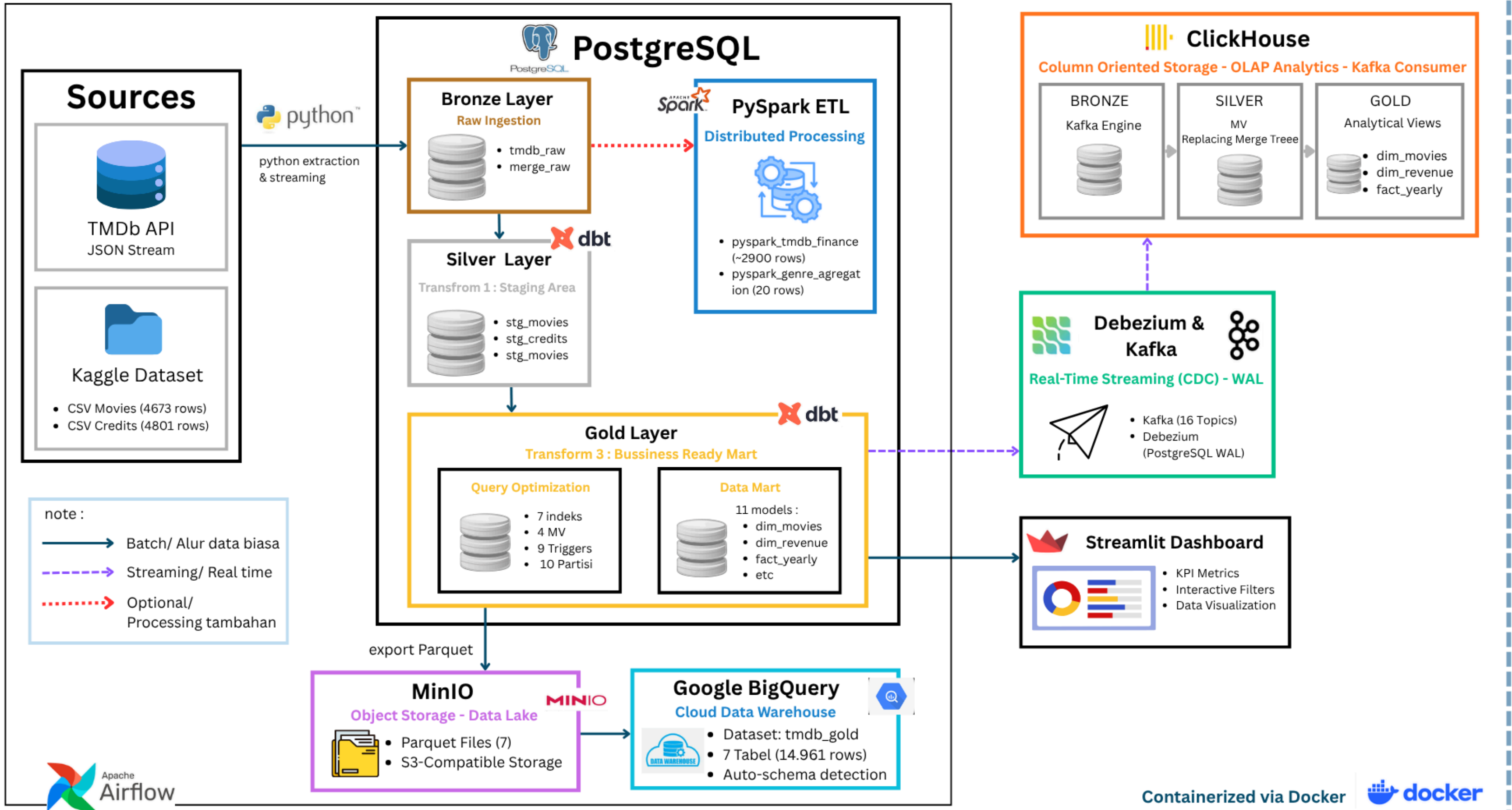
3 Kaggle – tmdb_5000_credits.csv

- Data type : CSV
- Source : [klik ini](#)
- 4801 rows
- Cast dan crew per film

Tools



End-To-End Data Engineering Pipeline : TMDB MOVIE



Step 1: Bronze Layer

Why?

Data raw disimpan tanpa transformasi apapun.

Docker-Setup

1. postgres:13 (Docker)
2. dpage/pgadmin4 (Docker UI)

Data-Sources

1. TMDb API (80 rows)
2. credits raw csv file from Kaggle (4.803 rows)
3. movies raw csv file from Kaggle (4.807)

Output-Tables

1. **tmdb_raw** (15 Kolom, 80 rows)
2. **merge_raw**, gabungan 2 file csv Kaggle (22 kolom, 4.803 baris)

Load tabel tmdb_raw dari API ke pgAdmin

The screenshot shows the pgAdmin interface with the 'tmdb_raw' table selected. The table has 15 columns: adult, backdrop_path, genre_ids, id, original_language, original_title, overview, popularity, poster_path, release_date, title, video, vote_average, vote_count, and category. The data is displayed in a table view showing rows 1 to 80.

	adult	backdrop_path	genre_ids	id	original_language	original_title	overview
1	false	/6yeVcxFRQj08vhr20IL6zbwm40.jpg	(28,878,53)	1265609	en	War Machine	On one last grueling mission dur
2	false	/u6YE4nEWOYyoghPJaNy15GHxpu.jpg	(28,80,53)	1290821	en	Shelter	A man living in self-imposed exil
3	false	/zpEWFNqoN8Qg1SdMMHma0y0BT...	(53,80,35)	1054867	en	One Battle After Another	Washed-up revolutionary Bob ex
4	false	/hYgUkH7TusddH8teJ53l6gFOWR.jpg	(28,53)	799882	en	The Bluff	When her tranquil life on a remot
5	false	/1x9e0qWonw634NhisRdvneeqvN...	(10749,18)	1523145	ru	Трое cepгue бьют пасторо	High school student Polina is sa
6	false	/lgoja3xMoJZbYmHfcQcJAEMWHJ...	(16,35,12,10751,96...	1084242	en	Zootopia 2	After cracking the biggest case i
7	false	/2RtLuffizGWWiH8IAe0o0YHmx.jpg	(16,10751,878,35)	1327819	en	Hoppers	Scientists have discovered how t
8	false	/u9E0R1nvkmdWu3t7OpIjoPywa.jpg	(12,53,878)	840464	en	Greenland 2: Migration	Having found the safety of the G
9	false	/nAuGnGH0sfzUTh20zgmRwKu3J...	(27,28,53)	1233413	en	Sinners	Trying to leave their troubled liv
10	false	/7Hkpc11uQfkw0Y8BRUyn1fsKqE.jpg	(878,28,53)	1236153	en	Mercy	In the near future, a detective sta
11	false	/tadE0v4Np4ndW4P0vW4E0AYW1...	(758,53)	1088474	en	Matfina	A drifter with a mysterious past

Hasil ingestion dari 2 file CSV ke tabel merge_raw

The screenshot shows the pgAdmin interface with the 'merge_raw' table selected. The table has 22 columns: budget, genres, homepage, id, keywords, original_language, original_title, overview, popularity, production_companies, production_countries, release_date, revenue, runtime, spoken_languages, status, tagline, title, vote_average, vote_count, and cast. The data is displayed in a table view showing rows 1 to 8.

	budget	genres	homepage
1	237000000	['id': 28, 'name': 'Action'], ['id': 12, 'name': 'Adventure'], ['id': 14, 'name': 'Fantasy'], ['id': 878, 'name': 'Science Fic...	http://www.avatarm
2	300000000	['id': 12, 'name': 'Adventure'], ['id': 14, 'name': 'Fantasy'], ['id': 28, 'name': 'Action']	http://disney.go.com
3	245000000	['id': 28, 'name': 'Action'], ['id': 12, 'name': 'Adventure'], ['id': 80, 'name': 'Crime']	http://www.sonypic
4	250000000	['id': 28, 'name': 'Action'], ['id': 80, 'name': 'Crime'], ['id': 18, 'name': 'Drama'], ['id': 53, 'name': 'Thriller']	http://www.thedark
5	260000000	['id': 28, 'name': 'Action'], ['id': 12, 'name': 'Adventure'], ['id': 878, 'name': 'Science Fiction']	http://movies.disne
6	41000000	['id': 35, 'name': 'Comedy']	http://www.loveyou
7	258000000	['id': 14, 'name': 'Fantasy'], ['id': 28, 'name': 'Action'], ['id': 12, 'name': 'Adventure']	http://www.sonypic
8	260000000	['id': 16, 'name': 'Animation'], ['id': 10751, 'name': 'Family']	http://disney.go.com

Step 2 : Silver Layer

Purpose

Clean the data before the analytics process.

3 Staging Models :

stg_movies.sql

Source : merge_raw

- 1. Handle 2 date formats
- 2. Parse JSON columns
- 3. Normalize popularity & vote_avg

stg_credits.sql

Source : merge_raw

Extract lead actor, top 3 cast, director, screenplay writer, producer, cinematographer, and music composer from JSON columns.

stg_tmdbapi.sql

Source : tmdb_raw

- 1. Fix date format
- 2. Handle genre_ids as a PostgreSQL native array

stg_credits.sql

	movie_id bigint	title text	top_3_cast text	lead_actor text	total_cast_count integer
1	19995	Avatar	Sam Worthington, Zoe Saldana, Sigourney Weaver	Sam Worthington	
2	285	Pirates of the Caribbean: At World's End	Johnny Depp, Orlando Bloom, Keira Knightley	Johnny Depp	
3	206647	Spectre	Daniel Craig, Christoph Waltz, Léa Seydoux	Daniel Craig	
4	49026	The Dark Knight Rises	Christian Bale, Michael Caine, Gary Oldman	Christian Bale	
5	49529	John Carter	Taylor Kitsch, Lynn Collins, Samantha Morton	Taylor Kitsch	
6	16538	I Love You, Man	Paul Rudd, Jason Segel, Rashida Jones	Paul Rudd	
7	559	Spider-Man 3	Tobey Maguire, Kirsten Dunst, James Franco	Tobey Maguire	
8	38757	Tangled	Zachary Levi, Mandy Moore, Donna Murphy	Zachary Levi	
9	99861	Avengers: Age of Ultron	Robert Downey Jr., Chris Hemsworth, Mark Ruffalo	Robert Downey Jr.	
10	767	Harry Potter and the Half-Blood Prince	Daniel Radcliffe, Rupert Grint, Emma Watson	Daniel Radcliffe	
11	209112	Batman v Superman: Dawn of Justice	Ben Affleck, Henry Cavill, Gal Gadot	Ben Affleck	
12	1452	Superman Returns	Brandon Routh, Kevin Spacey, Kate Bosworth	Brandon Routh	
13	10764	Quantum of Solace	Daniel Craig, Olga Kurylenko, Mathieu Amalric	Daniel Craig	
14	58	Pirates of the Caribbean: Dead Man's Chest	Johnny Depp, Orlando Bloom, Keira Knightley	Johnny Depp	

stg_movies.sql

	movie_id bigint	title text	original_language text	overview text
1	19995	Avatar	en	In the 22nd century, a paraplegic Marine is dispatched to the moon Pandora on a un...
2	285	Pirates of the Caribbean: At World's End	en	Captain Barbossa, long believed to be dead, has come back to life and is headed to...
3	206647	Spectre	en	A cryptic message from Bond's past sends him on a trail to uncover a sinister organ...
4	49026	The Dark Knight Rises	en	Following the death of District Attorney Harvey Dent, Batman assumes responsibilit...
5	49529	John Carter	en	John Carter is a war-weary, former military captain who's inexplicably transported to...
6	16538	I Love You, Man	en	Peter Klaven is a successful real estate agent who, upon getting engaged to the wo...
7	559	Spider-Man 3	en	The seemingly invincible Spider-Man goes up against an all-new crop of villain - inc...
8	38757	Tangled	en	When the kingdom's most wanted-and most charming-bandit Flynn Rider hides out...
9	99861	Avengers: Age of Ultron	en	When Tony Stark tries to jumpstart a dormant peacekeeping program, things go aw...
10	767	Harry Potter and the Half-Blood Prince	en	As Harry begins his sixth year at Hogwarts, he discovers an old book marked as 'Pr...
11	209112	Batman v Superman: Dawn of Justice	en	Fearing the actions of a god-like Super Hero left unchecked, Gotham City's own for...
12	1452	Superman Returns	en	Superman returns to discover his 5-year absence has allowed Lex Luthor to walk fre...
13	10764	Quantum of Solace	en	Quantum of Solace continues the adventures of James Bond after Casino Royale. B...
14	58	Pirates of the Caribbean: Dead Man's Chest	en	Captain Jack Sparrow works his way out of a blood debt with the ghostly Davay...

Step 2 : Gold Layer – Schema : silver_mart

Purpose

Data ready for analytics and dashboard use.

Kategori Performa dari tabel dim_revenue

Label	Keterangan
Mega Blockbuster	Film sukses besar, balik modal ≥ 3x lipat
Blockbuster	Film sukses, balik modal ≥ 2x lipat
Profitable	Untung tapi tidak signifikan
Break Even	Tidak untung tidak rugi
Loss	Film merugi

	table_schema name	table_name name
1	public	merge_raw
2	public	tmdb_raw
3	silver_mart	dim_actor
4	silver_mart	dim_company
5	silver_mart	dim_country
6	silver_mart	dim_directors
7	silver_mart	dim_genres
8	silver_mart	dim_language
9	silver_mart	dim_movies
10	silver_mart	dim_revenue
11	silver_mart	dim_yearlysu...
12	silver_mart	fact_vs
13	silver_mart	fact_yearly

11 Mart Models

1. **dim_movies**
Complete master film table, merged from 3 sources

2. **dim_genres**
Performace summary by genre

3. **dim_directors**
Director career profile and performance

4. **dim_actor**
Career profile for lead and supporting actors

5. **dim_country**
Film summary (revenue, rating, etc.) per country

6. **dim_language**
Film analysis by language
7. **dim_company**
Production house profile and analysis

8. **dim_revenue**
Financial summary per film (ROI, budget, profit, etc.)

9. **fact_vs**
Reconciliation of 2 data sources (CSV vs API)

10. **fact_yearly**
Industry trends per year and per genre

11. **dim_yearlysum**
Film summary grouped by year

Step 3 : Query Optimization

Step by Step :

1 EXPLAIN – Query Performa Analysis

Run on **dim_movies** and **dim_revenue** as a baseline before optimization. Then **created 7 indexes**.

	QUERY PLAN
	text
1	Sort (cost=13.24..13.31 rows=27 width=36)
2	Sort Key: release_year
3	-> HashAggregate (cost=12.26..12.60 rows=27 width=36)
4	Group Key: release_year
5	-> Seq Scan on fact_yearly (cost=0.00..10.45 rows=363 w...
6	Filter: ((release_year >= 1990) AND (release_year <= 20...

Proven that indexes work – **COST drops drastically.**

	QUERY PLAN
	text
1	Limit (cost=79.52..79.53 rows=1 width=31)
2	-> Sort (cost=79.52..79.53 rows=1 width=31)
3	Sort Key: revenue DESC
4	-> Bitmap Heap Scan on dim_revenue (cost=8.69..79.51 rows=1 width=31)
5	Recheck Cond: (primary_genre = 'Action'::text)
6	Filter: (vote_average > 7.0)
7	-> Bitmap Index Scan on idx_dim_revenue_genre (cost=0.00..8.69 rows=588 widt...
8	Index Cond: (primary_genre = 'Action'::text)

2 PARTITION

The **fact_yearly_partisi** table is divided into partitions per Decade.

	decade	total_films	total_revenue	avg_roi_pct	revenue_growth_pct
	integer	bigint	numeric	numeric	numeric
1	1990	540	60946155147	867.37	[null]
2	2000	1335	155003313780	4541.52	154.3
3	2010	947	148615359698	122918.97	-4.1

Step 3 : Query Optimization

3 Materialized View + Auto Refresh Trigger

Created as aggregation caches for the dashboard, automatically triggering MV refresh without manual intervention.

MV 1 : Performa by country : dominant country per genre/profit, favorite genre per country

The screenshot shows a database query editor with the following SQL query: `SELECT * FROM public.mv_country_performance`. The data output table has the following columns: `primary_country` (text), `total_films` (bigint), `total_revenue` (numeric), `total_profit` (numeric), `avg_roi_pct` (numeric), `avg_rating` (numeric), `most_common_genre` (text), and `top_grossing_film` (text). The table displays three rows of data.

	primary_country	total_films	total_revenue	total_profit	avg_roi_pct	avg_rating	most_common_genre	top_grossing_film
1	United States of Ameri...	3102	280520928071	189414222081	207.90	6.09	Comedy	Avatar
2	United Kingdom	374	36541235127	25848003976	241.72	6.46	Drama	Skyfall
3	Germany	200	16669142023	9338645341	127.39	6.23	Drama	Harry Potter and the Chamber of S

MV 2 : Financial per Genre per Decade — trending genres, etc.

The screenshot shows a database query editor with the following SQL query: `SELECT * FROM public.mv_revenue_summary`. The data output table has the following columns: `primary_genre` (text), `decade` (integer), `total_films` (bigint), `mega_blockbuster` (bigint), `blockbuster` (bigint), `loss` (bigint), `avg_roi_pct` (numeric), `total_revenue` (numeric), and `total_profit` (numeric). The table displays seven rows of data.

	primary_genre	decade	total_films	mega_blockbuster	blockbuster	loss	avg_roi_pct	total_revenue	total_profit
1	Action	2010	188	66	43	32	179.10	45854233587	30447602556
2	Adventure	2000	125	45	25	28	200.04	35464606380	24224115350
3	Action	2000	224	50	37	74	102.48	27386091171	15352691171
4	Comedy	2000	294	112	49	81	279.95	24723193779	16104347325
5	Drama	2000	322	102	43	118	401.28	22850141084	12930461632
6	Adventure	2010	61	17	20	10	173.35	19031318998	12291318998
7	Comedy	2010	175	71	31	34	229.22	16092294965	10495397279

MV 3 : PDirector Performance — most profitable directors, career duration, etc.

The screenshot shows a database query editor with the following SQL query: `SELECT * FROM public.mv_director_performance`. The data output table has the following columns: `director` (text), `total_films` (bigint), `avg_rating` (numeric), `total_box_office` (numeric), `total_profit` (numeric), `roi_pct` (numeric), `first_film_year` (integer), `latest_film_year` (integer), and `career_years` (integer). The table displays six rows of data.

	director	total_films	avg_rating	total_box_office	total_profit	roi_pct	first_film_year	latest_film_year	career_years
1	David Hand	3	0.70	535692636	531066213	11478.98	1937	1942	6
2	Alex Kendrick	4	0.68	78156869	76556869	4784.80	2003	2011	9
3	Victor Fleming	3	0.70	439294426	429890426	4571.36	1939	1944	6
4	Darren Lynn Bousman	5	0.62	456342667	432154541	1800.64	2005	2016	12
5	Guy Hamilton	6	0.65	507480445	478780445	1668.22	1964	1978	15
6	Henry Joost	3	0.61	432229120	402229120	1340.76	2011	2016	6

MV 4: Genre Summary — most profitable, most popular, annual trends

The screenshot shows a database query editor with the following SQL query: `SELECT * FROM public.mv_genre_summary`. The data output table has the following columns: `genre` (text), `total_films` (bigint), `avg_rating` (numeric), `avg_popularity` (numeric), `total_revenue` (numeric), `total_budget` (numeric), `total_profit` (numeric), `avg_runtime_minutes` (numeric), `earliest_release` (integer), `latest_release` (integer), and `top_grossing_film` (text). The table displays ten rows of data.

	genre	total_films	avg_rating	avg_popularity	total_revenue	total_budget	total_profit	avg_runtime_minutes	earliest_release	latest_release	top_grossing_film
	Action	588	6.07	34.63	91459088189	34233272285	57225815904	111.0	1930	2016	Avata
	Adventure	288	6.39	44.60	70872316207	21705338117	49166978090	116.0	1939	2016	Capta
	Drama	747	6.66	22.50	54101757502	19691630594	34410126908	121.0	1916	2016	Titani
	Comedy	634	6.06	20.32	53037118804	17399824524	35637294280	103.0	1933	2016	Shrek
	Animation	99	6.37	41.69	29595221289	8185432353	21409788936	91.0	1940	2016	Froze
	Fantasy	93	6.25	40.29	17057198167	5930438831	11126759336	108.0	1937	2016	Spide
	Science Ficti...	79	6.34	49.13	16176309703	4631307003	11545002700	113.0	1968	2016	The A

Step 4 : PySpark ETL

Concept

PySpark is suited for production pipelines handling large data, with lazy execution, petabyte-scale capability, and spark.sql() support. Distributed in-memory processing, not just pandas.

Alur ETL dengan PySpark

1

EXTRACT

merge_raw table from PostgreSQL via JDBC (4,803 rows, 22 columns)

2

TRANSFORM 1 – JSON Column Extraction

- genres → primary_genre
- keywords → primary_keyword
- production_companies → primary_company
- production_countries → primary_country
- spoken_languages → primary_language
- crew → primary_director
- cast → lead_actor

4

TRANSFORM 3 – Agregasi per genre

```
2 FROM public.pyspark_genre_aggregation
3 ORDER BY total_revenue DESC;
```

	primary_genre text	total_films bigint	total_revenue bigint	avg_roi_pct double precision
1	Action	588	91459088189	187.51
2	Adventure	288	70872316207	471.22
3	Drama	747	54101757502	1138398.15
4	Comedy	634	53037118804	425.95
5	Animation	99	29595221289	620.67
6	Fantasy	93	17057198167	370.35
7	Science Fiction	79	16176309703	418.92

3

TRANSFORM 2 – New Column Calculations

- profit = revenue - budget
- roi_pct = (revenue - budget) / budget × 100
- performance_label: Mega Blockbuster / Blockbuster / etc.
- popularity_tier: Viral (>100) / Popular (>50) / Known (>20) / Niche

```
2 FROM public.pyspark_tmdb_finance
3 GROUP BY performance_label
4 ORDER BY total DESC;
```

	performance_label text	total bigint
1	Mega Blockbuster	2313
2	Loss	1326
3	Profitable	629
4	Blockbuster	533
5	Break Even	2

Step 5 : Kafka + Debezium CDC

Concept

Builds a Change Data Capture (CDC) system that captures every data change in PostgreSQL in real time and streams it to Kafka.

Step-by-Step

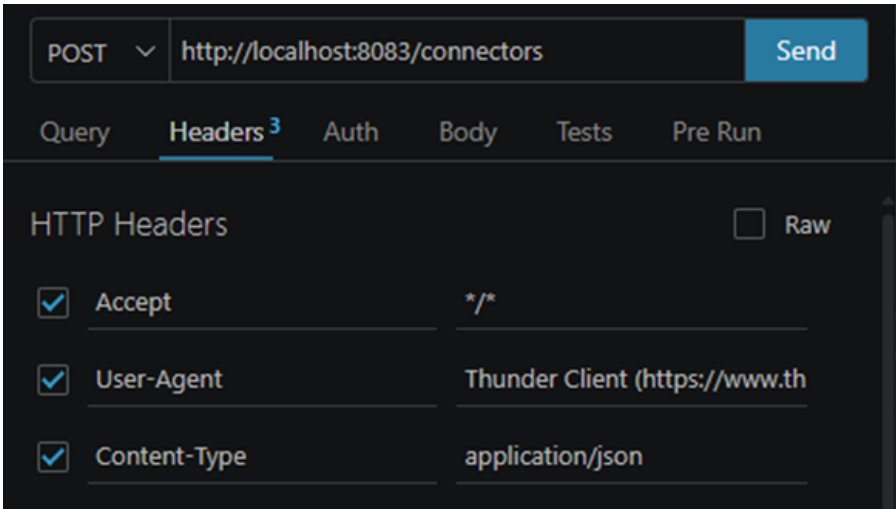
1 Kafka Stack Components

Container	Image	Port	Fungsi
tmdb_zookeeper	confluentinc/cp-zookeeper:7.4.0	2181	Koordinasi Kafka cluster
tmdb_kafka	confluentinc/cp-kafka:7.4.0	29092:29092	Message broker, penyimpan event stream
tmdb_debezium	debezium/connect:2.4	8083:8083	CDC connector PostgreSQL → Kafka
tmdb_kafka_man	hlebalbau/kafka-manager	9000:9000	Web UI monitoring Kafka

2 Configure PostgreSQL to WAL Logical

3 Create Debezium Connector

Connector created via HTTP POST to the Debezium REST API using Thunder Client



4 Output Kafka Topics di Kafka UI

Debezium automatically creates 1 topic per table

Topic Kafka	Sumber Tabel
tmdb.silver_mart.dim_actor	silver_mart.dim_actor
tmdb.silver_mart.dim_company	silver_mart.dim_company
tmdb.silver_mart.dim_country	silver_mart.dim_country
tmdb.silver_mart.dim_directors	silver_mart.dim_directors
tmdb.silver_mart.dim_genres	silver_mart.dim_genres
tmdb.silver_mart.dim_language	silver_mart.dim_language
tmdb.silver_mart.dim_movies	silver_mart.dim_movies
tmdb.silver_mart.dim_revenue	silver_mart.dim_revenue
tmdb.silver_mart.dim_yearlysum	silver_mart.dim_yearlysum
tmdb.silver_mart.fact_vs	silver_mart.fact_vs
tmdb.silver_mart.fact_yearly	silver_mart.fact_yearly
+ 5 internal Kafka topics	__consumer_offsets, connect_configs, dll

(Screenshot Kafka UI)

Topic	# Partitions	# Brokers	Brokers Spread %	Brokers Skew %	Brokers Leader Skew %	# Replicas	Under Replicated %
__confluent.support.metrics	1	1	100	0	0	1	0
__consumer_offsets	50	1	100	0	0	1	0
connect-status	5	1	100	0	0	1	0
connect_configs	1	1	100	0	0	1	0
connect_offsets	25	1	100	0	0	1	0
tmdb.silver_mart.dim_actor	1	1	100	0	0	1	0
tmdb.silver_mart.dim_company	1	1	100	0	0	1	0
tmdb.silver_mart.dim_country	1	1	100	0	0	1	0
tmdb.silver_mart.dim_directors	1	1	100	0	0	1	0
tmdb.silver_mart.dim_genres	1	1	100	0	0	1	0
tmdb.silver_mart.dim_language	1	1	100	0	0	1	0
tmdb.silver_mart.dim_movies	1	1	100	0	0	1	0
tmdb.silver_mart.dim_revenue	1	1	100	0	0	1	0
tmdb.silver_mart.dim_yearlysum	1	1	100	0	0	1	0
tmdb.silver_mart.fact_vs	1	1	100	0	0	1	0
tmdb.silver_mart.fact_yearly	1	1	100	0	0	1	0

Step 6 : ClickHouse OLAP

Arsitektur 3 Layer di ClickHouse

Purpose

OLAP (column-oriented), aggregation analytics faster than PostgreSQL (row-oriented).

1 BRONZE: Kafka Engine Table

- Directly consumes from Kafka topic.
- ENGINE = Kafka SETTINGS kafka_broker_list = 'tmdb-kafka:9092'

2 SILVER: ReplacingMergeTree Table

- Stores final data, prevents duplicates.
- ENGINE = ReplacingMergeTree (insert_time)

3 GOLD: View

- Adds calculated columns: profit, performance_label, popularity_tier.
- WHERE budget > 0 AND revenue > 0

Output

9 Total Objects in ClickHouse

Nama Objek	Tipe	Engine / Keterangan
dim_movies	Base Table	ReplacingMergeTree — data final silver layer
dim_movies_queue	Kafka Queue	Kafka Engine — consume dari topic
dim_movies_mv	Materialized View	Jembatan queue → base table (otomatis)
dim_revenue	Base Table	ReplacingMergeTree
dim_revenue_queue	Kafka Queue	Kafka Engine
dim_revenue_mv	Materialized View	Otomatis
fact_yearly	Base Table	ReplacingMergeTree
fact_yearly_queue	Kafka Queue	Kafka Engine
fact_yearly_mv	Materialized View	Otomatis

Tabel	Jumlah Rows	Keterangan
dim_movies	4802 rows	Data mengalir dari Kafka CDC
dim_revenue	3229 rows	Film dengan data keuangan lengkap
fact_yearly	363 rows	Agregasi per tahun per genre

Data Validation Results in ClickHouse

<pre>SELECT COUNT(*) FROM dim_revenue Query id: 1ef6ef76-2 1. COUNT() 3229 1 row in set. Elapse</pre>	<pre>SELECT COUNT(*) FROM fact_yearly Query id: 3d81fc87- 1. COUNT() 363 1 row in set. Elapse</pre>	<pre>SELECT COUNT(*) FROM dim_movies Query id: 533c5984-0 1. COUNT() 4802 1 row in set. Elapse</pre>
---	---	--

Step 7 : MinIO Object Storage

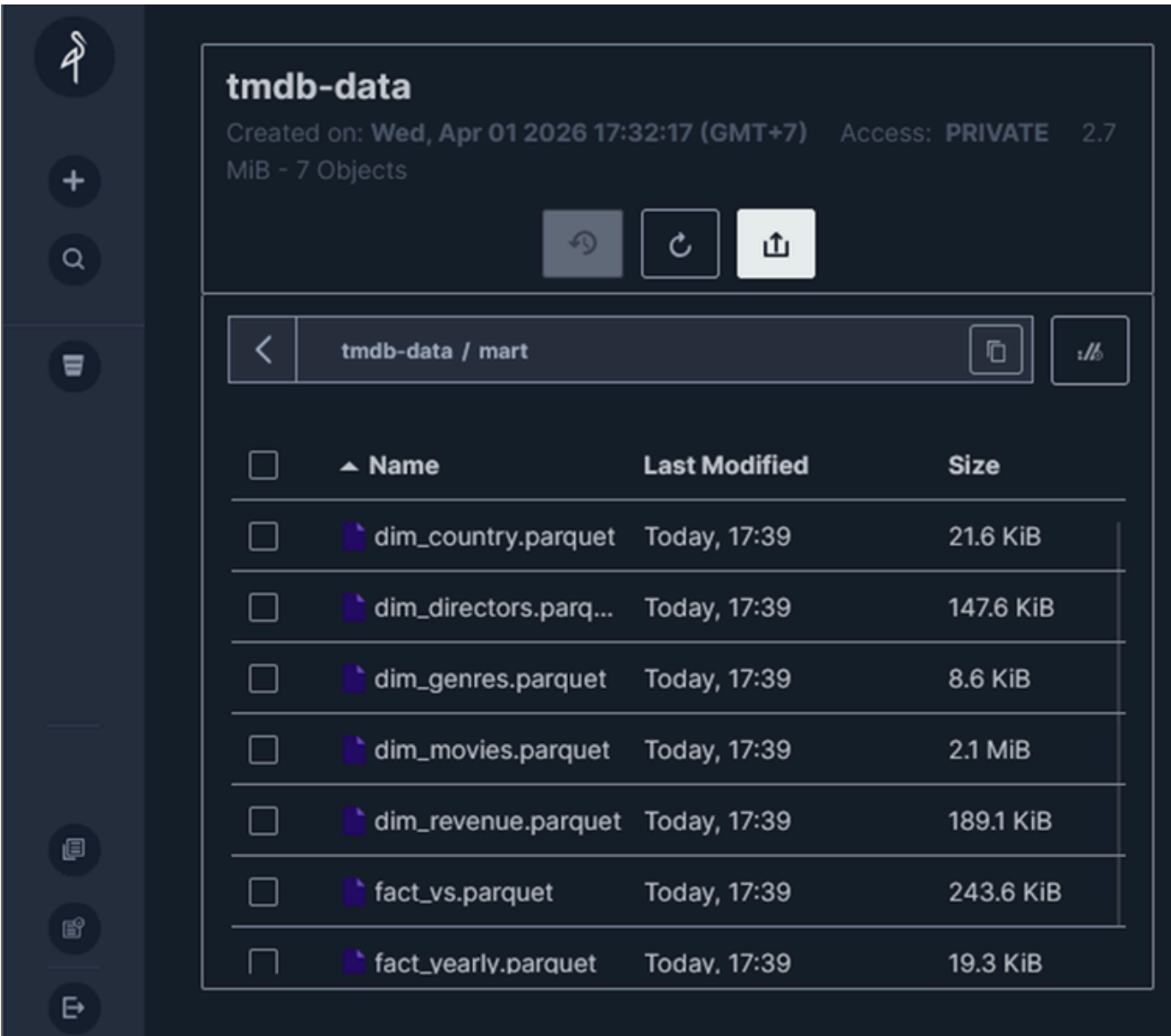
Concept

S3-compatible storage, a modern replacement for HDFS. Used to store mart layer results from PostgreSQL in Parquet format. Stores data outside the database for backup, sharing, or further processing purposes.

Output

Upload Results — Bucket: `tmdb-data/mart`
= Total: 7 Parquet files, 2.7 MiB

File Parquet	Jumlah Rows	Ukuran
dim_movies.parquet	4.802	2.1 MiB
dim_directors.parquet	2.349	147.6 KiB
fact_vs.parquet	4.867	243.6 KiB
dim_revenue.parquet	3.229	189.1 KiB
dim_country.parquet	70	21.6 KiB
fact_yearly.parquet	363	19.3 KiB
dim_genres.parquet	19	8.6 KiB



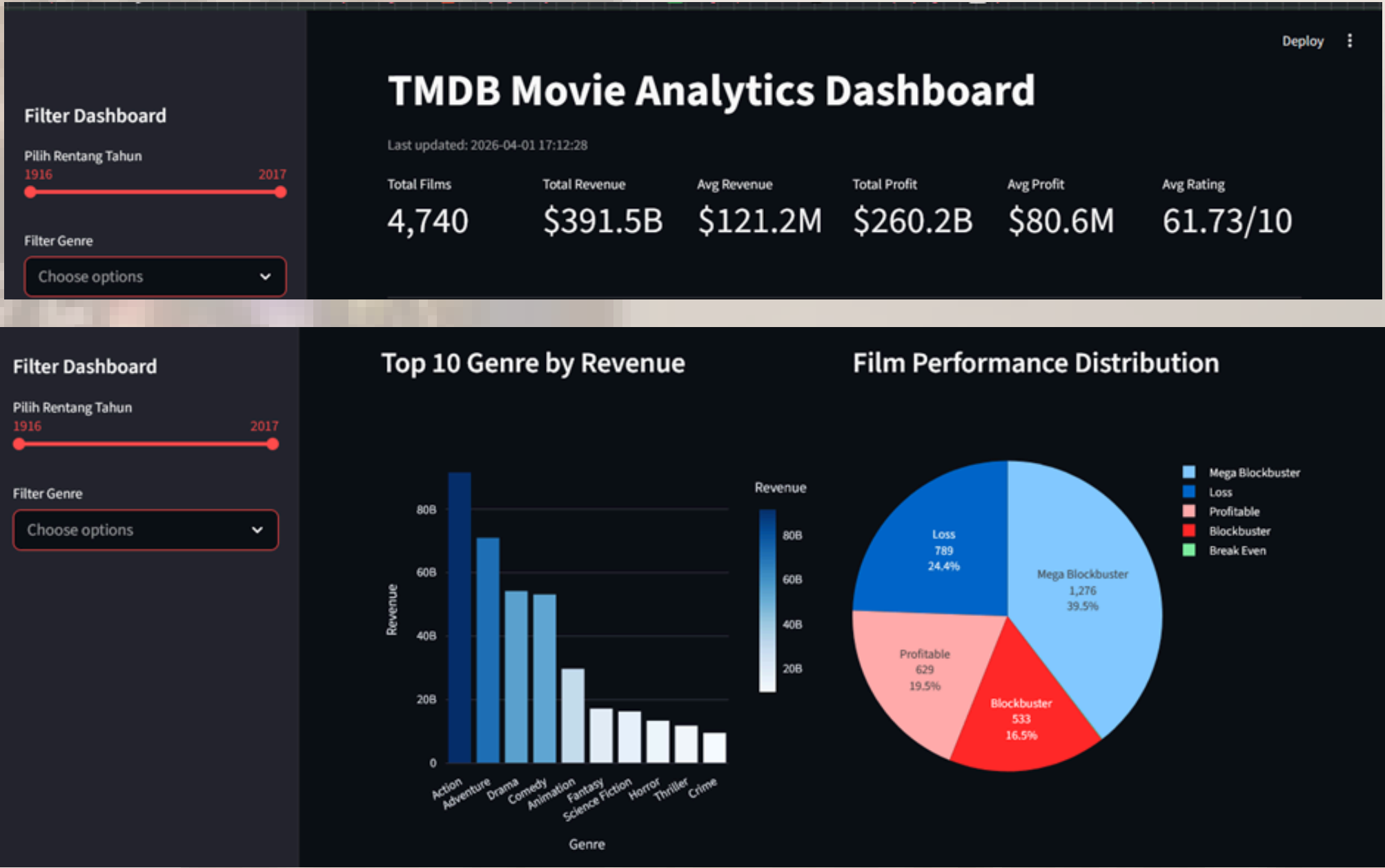
Step 8 : Streamlit Dashboard: Visualisasi Data

Purpose An interactive dashboard built with Streamlit (Python library), connected directly to data in PostgreSQL. The dashboard displays KPI scorecards, charts, and tables with interactive filters.

Dashboard Features

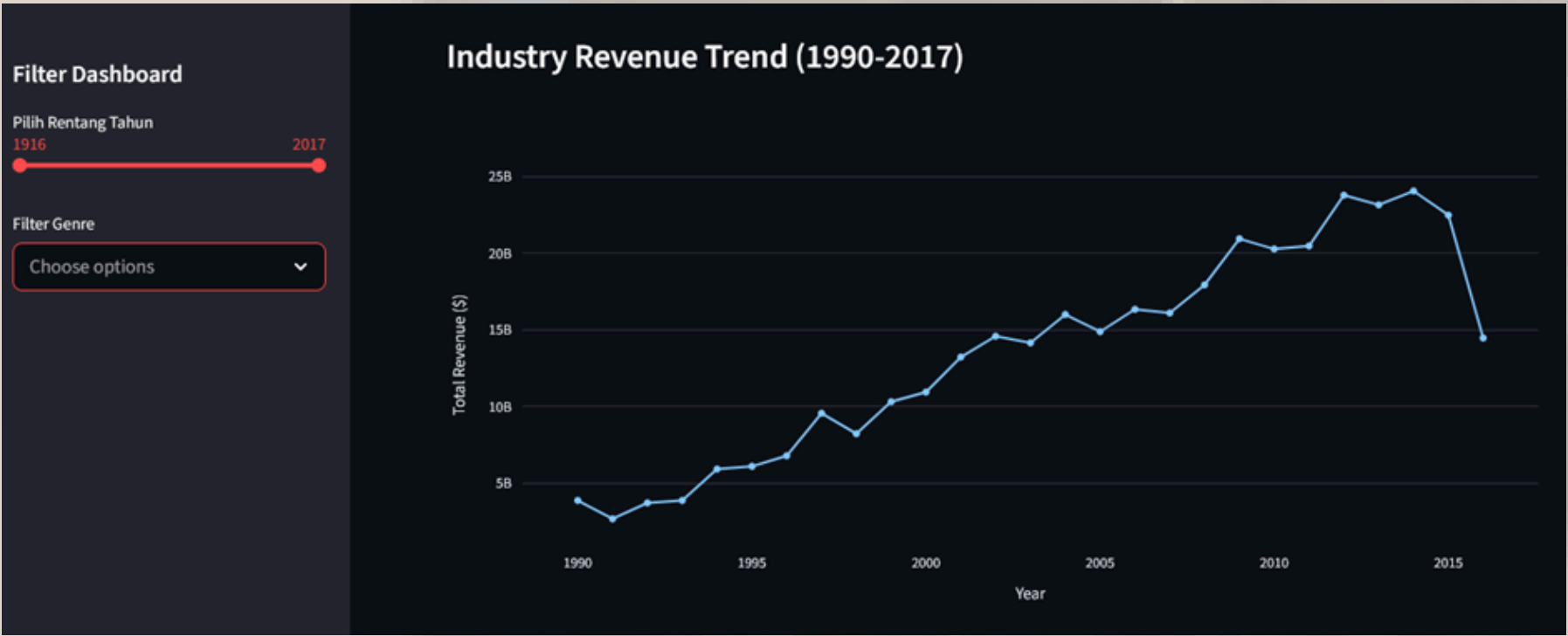
Fitur	Keterangan
Filter Rentang Tahun	Slider untuk memilih tahun rilis (misal: 1959–2004)
Filter Genre	Multi-select untuk memfilter genre (Comedy, Adventure, dll)
Scorecard KPI	Total Films, Total Revenue, Avg Revenue, Total Profit, Avg Profit, Avg Rating
Top 10 Genre by Revenue	Bar chart revenue per genre
Film Performance Distribution	Pie chart distribusi Mega Blockbuster / Blockbuster / Profitable / Loss
Top 20 Highest Revenue Films	Tabel detail dengan kolom Title, Genre, Lead Actor, Director, Revenue, Performance

Screenshot Dashboard Result

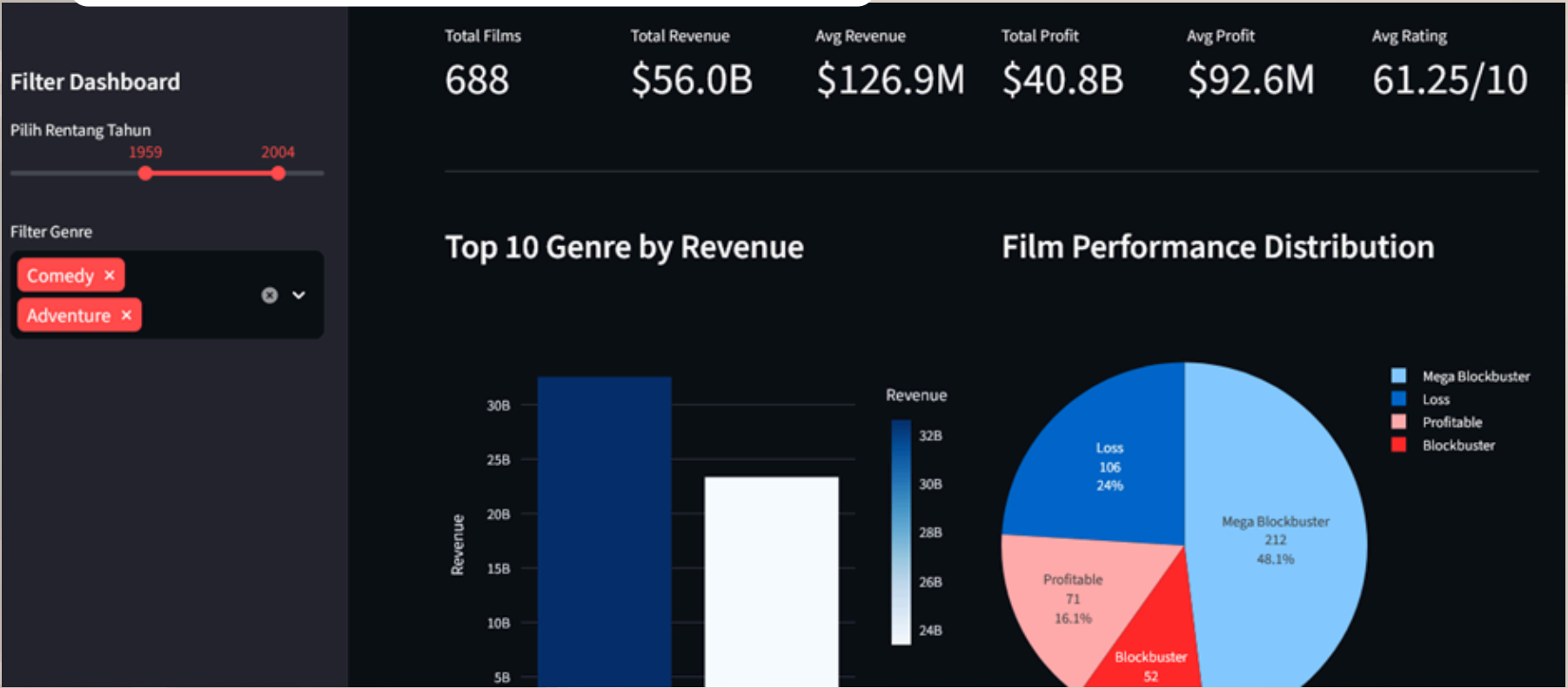


Step 8 : Streamlit Dashboard: Visualisasi Data

Hasil Screenshot Dashboard



- Dashboard with year filter 1959–2004 and genre Comedy and Adventure activated



Filter Dashboard

Pilih Rentang Tahun
1916 2017

Filter Genre
Choose options

Top 20 Highest Revenue Films

	Title	Genre	Lead Actor	Director	Revenue	Profit	Performance	Ratin
0	Avatar	Action	Sam Worthington	James Cameron	\$2,787,965,087	\$2,550,965,087	Mega Blockbuster	7.2/1
1	Titanic	Drama	Kate Winslet	James Cameron	\$1,845,034,188	\$1,645,034,188	Mega Blockbuster	7.5/1
2	The Avengers	Science Fiction	Robert Downey Jr.	Joss Whedon	\$1,519,557,910	\$1,299,557,910	Mega Blockbuster	7.4/1
3	Jurassic World	Action	Chris Pratt	Colin Trevorrow	\$1,513,528,810	\$1,363,528,810	Mega Blockbuster	6.5/1
4	Furious 7	Action	Vin Diesel	James Wan	\$1,506,249,360	\$1,316,249,360	Mega Blockbuster	7.3/1
5	Avengers: Age of Ultron	Action	Robert Downey Jr.	Joss Whedon	\$1,405,403,694	\$1,125,403,694	Mega Blockbuster	7.3/1
6	Frozen	Animation	Kristen Bell	Chris Buck	\$1,274,219,009	\$1,124,219,009	Mega Blockbuster	7.3/1
7	Iron Man 3	Action	Robert Downey Jr.	Shane Black	\$1,215,439,994	\$1,015,439,994	Mega Blockbuster	6.8/1
8	Minions	Family	Sandra Bullock	Kyle Balda	\$1,156,730,962	\$1,082,730,962	Mega Blockbuster	6.4/1
9	Captain America: Civil Wi	Adventure	Chris Evans	Anthony Russo	\$1,153,304,495	\$903,304,495	Mega Blockbuster	7.1/1

Filter Dashboard

Pilih Rentang Tahun
1959 2004

Filter Genre
Comedy x
Adventure x

Top 20 Highest Revenue Films

	Title	Genre	Lead Actor	Director	Revenue	Profit	Performance	Ratin
0	The Lord of the Rings: The Re	Adventure	Elijah Wood	Peter Jackson	\$1,118,411,000	\$851,475,550	Mega Blockbus	7.5/10
1	Harry Potter and the Philosop	Adventure	Daniel Radcliffe	Chris Columbus	\$976,475,550	\$851,475,550	Mega Blockbus	7.5/10
2	The Lord of the Rings: The Tw	Adventure	Elijah Wood	Peter Jackson	\$926,287,400	\$847,287,400	Mega Blockbus	8.0/10
3	Star Wars: Episode I - The Pha	Adventure	Liam Neeson	George Lucas	\$924,317,558	\$809,317,558	Mega Blockbus	6.3/10
4	Jurassic Park	Adventure	Sam Neill	Steven Spielberg	\$920,100,000	\$857,100,000	Mega Blockbus	7.6/10
5	Shrek 2	Adventure	Mike Myers	Andrew Adamson	\$919,838,758	\$769,838,758	Mega Blockbus	6.7/10
6	Harry Potter and the Chambe	Adventure	Daniel Radcliffe	Chris Columbus	\$876,688,482	\$776,688,482	Mega Blockbus	7.4/10
7	The Lord of the Rings: The Fel	Adventure	Elijah Wood	Peter Jackson	\$871,368,364	\$778,368,364	Mega Blockbus	8.0/10
8	Harry Potter and the Prisoner	Adventure	Daniel Radcliffe	Alfonso Cuarón	\$789,804,554	\$659,804,554	Mega Blockbus	7.7/10
9	Star Wars	Adventure	Mark Hamill	George Lucas	\$775,398,007	\$764,398,007	Mega Blockbus	8.1/10

Step 9 : Google BigQuery: Cloud Data Warehouse

Purpose Stores 7 Gold Layer tables from MinIO (Parquet format) into Google BigQuery as a cloud data warehouse – ready for analytics, BI tools, and data sharing.

Alur-Export

1. Read 7 Parquet files from MinIO bucket tmdb-data/mart/
2. Convert to DataFrame → Upload to BigQuery dataset tmdb_gold
3. Mode WRITE_TRUNCATE – automatically replaces old data on every run

Cek-Hasil:

<https://console.cloud.google.com/bigquery> → latihan-dsarea → tmdb_gold → Preview

Success 7/7 tables – executed via Airflow DAG task export_to_bigquery.

Screenshot BigQuery

The screenshot shows the Google BigQuery console interface. On the left, a sidebar lists the project 'latihan-dsarea' and its datasets, including 'tmdb_gold'. The 'dim_country' table is selected. The main panel displays the 'Schema' tab for this table, showing a list of fields with their names, types, and modes.

Field name	Type	Mode	Description	Key	Collation	De
primary_country	STRING	NULLABLE	-	-	-	-
total_films	INTEGER	NULLABLE	-	-	-	-
total_revenue	FLOAT	NULLABLE	-	-	-	-
avg_rating	FLOAT	NULLABLE	-	-	-	-
most_common_genre	STRING	NULLABLE	-	-	-	-
most_common_language	STRING	NULLABLE	-	-	-	-

Buttons at the bottom: Edit schema, View row access policies

Step 10 : Apache Airflow | DAG Orchestration

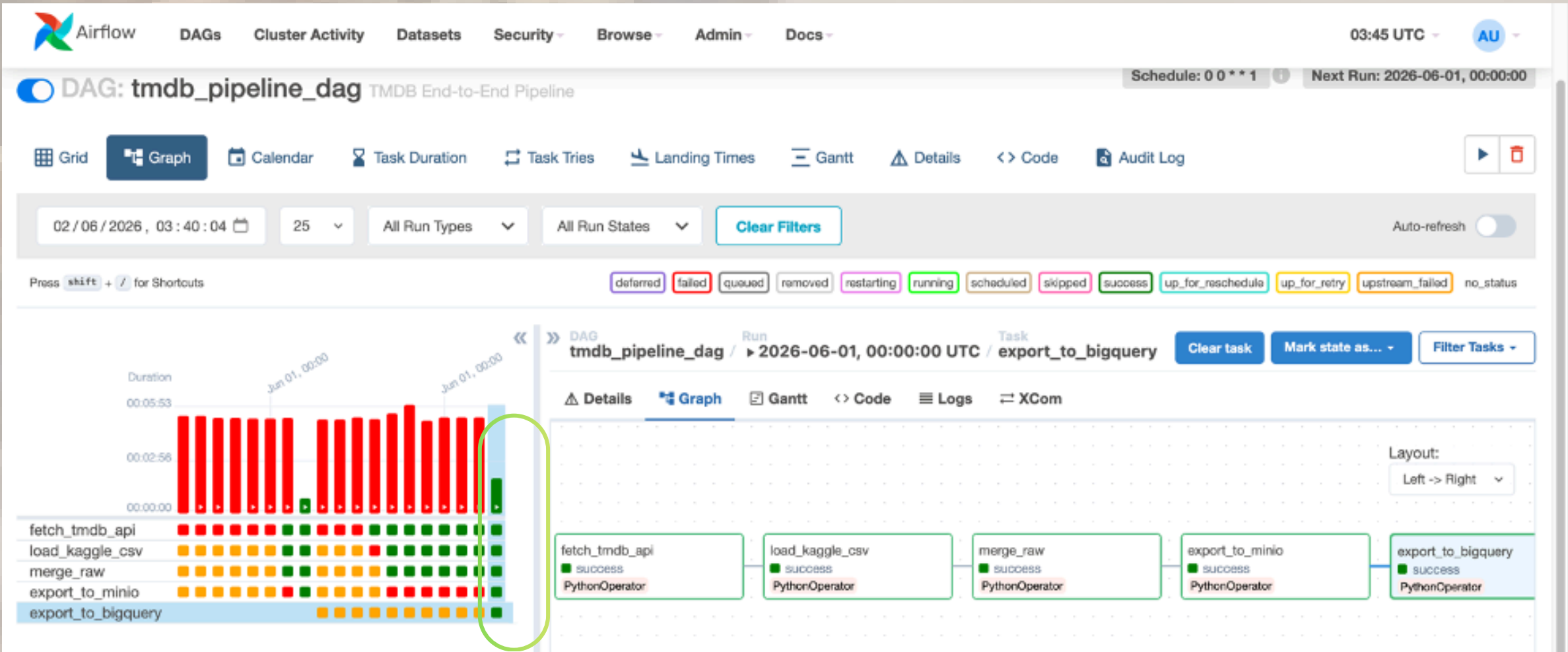
5 Tasks in the DAG (Sequential):

- Task 1 – fetch_tmdb_api: fetch_tmdb_api.py, load_kaggle_csv.py, merge_raw.py – fetch data from TMDb API + load Kaggle CSV to PostgreSQL
- Task 2 – load_kaggle_csv: Load Kaggle CSV files (tmdb_5000_movies + tmdb_5000_credits) to PostgreSQL
- Task 3 – merge_raw: Merge csv_movies_raw + csv_credits_raw into the merge_raw table (4,803 rows)
- Task 4 – export_to_minio: Export 7 mart tables from schema silver_mart_silver_mart → MinIO in Parquet format
- Task 5 – export_to_bigquery: Read 7 Parquet files from MinIO → Upload to Google BigQuery dataset tmdb_gold

Purpose

Automates & schedules the entire batch pipeline from ingestion → dbt → PySpark → MinIO → BigQuery in a single DAG.

Screenshot Airflow DAG



Conclutions

Step 1 - 8 : Step Pipeline

Step	Kategori	Nama Step	Tools / Teknologi	Main Output
1	Data Source	Data Source	TMDB API	Raw JSON data film
2	Data Ingestion	Ingestion	Python (Requests, JSON)	Raw data retrieved
3	Streaming	Message Broker	Apache Kafka	Streaming data (topics Kafka)
4	Storage (Bronze)	Storage (Raw)	MinIO	JSON / Parquet files (raw layer)
5	Processing (Silver)	Processing	PySpark	Transformed & cleaned data
6	Data Warehouse	Data Warehouse	ClickHouse	Analytics ready table
7	Transformation	Transformation	dbt	Dim & fact models (gold layer)
8	BI / Visualization	Visualization	Streamlit	Interactive Dashboard

Project Achievements

- **End-to-end pipeline without additional orchestration** – The entire flow from raw data to dashboard runs in an integrated manner. Without Airflow – every step is connected and produces valid output.
- **Hybrid architecture: batch + streaming** – Batch via dbt & PySpark, real-time via Kafka + Debezium CDC – the same pattern used in production-scale pipelines.
- **Analytics-ready data, good performance, efficient storage** – Fast queries via index + partition + MV in PostgreSQL & ClickHouse OLAP. Compact storage: 4,803 films stored in just 2.7 MiB in Parquet format.